What Skills Claim and What Skills Do: Detecting Intent-Behavior Misalignment in Agent Skill Ecosystems

I. INTRODUCTION

Agent skills extend the capabilities of large language model agents by combining declarative content, such as `SKILL.md`, `README`, and natural-language instructions, with executable content, including inline code, source files, and auxiliary scripts [1]. Declarative content specifies intended functionality, usage, and applicable scenarios, whereas executable content determines runtime behavior. Misalignment arises when implementations evolve without corresponding documentation updates or when malicious developers conceal sensitive operations through vague descriptions, functional disguises, or hidden instructions.For example, a skill presented as a file downloader may also access environment variables, transmit data externally, or modify system configurations [2]. Such intent–behavior misalignment undermines skill trustworthiness and may cause agents to perform unexpected or unauthorized actions [3]. Existing declaration–implementation consistency methods remain inadequate for agent skills. They typically analyze isolated sources, such as comments, application descriptions, permissions, or documentation, rather than jointly reasoning over heterogeneous descriptive and executable content [4]. Moreover, lexical matching, predefined rules, and semantic similarity cannot reliably determine whether an observed behavior functionally fulfills a declared intent. Source-level behavior may also emerge across multiple functions and execution paths, making isolated code-unit analysis insufficient. Consequently, detecting and explaining intent–behavior misalignment remains a key challenge in agent skill analysis.

II. CHALLENGES

Challenge 1: Multi-source information obscures skill boundaries. A skill's functionality is distributed across natural-language descriptions, instructions, source code, and auxiliary resources. Descriptions may omit executed operations, while some capabilities appear only in the implementation. Jointly interpreting these sources and distinguishing declared from executed functionality is therefore a fundamental challenge for intent–behavior misalignment detection.

Challenge 2: Functional fulfillment is difficult to determine from semantic relatedness. Skill declarations and behaviors do not follow a one-to-one mapping. Semantic similarity does not guarantee functional fulfillment, and lexical differences do not necessarily imply inconsistency. Keywords and surface-level similarity are therefore insufficient for determining whether execution results satisfy declared intents or for distinguishing unimplemented intents, undeclared behaviors, and semantically divergent behaviors.

Challenge 3: Risky behaviors may emerge from cross-unit execution. High-risk behaviors often arise through interactions among multiple functions, files, or execution steps and may remain hidden in isolated code fragments. For example, individually benign operations such as data reading, format conversion, and network transmission may collectively enable undisclosed data exfiltration. Recovering such composite behaviors and relating them to declared intents remains a key challenge.

III. TECHNICAL REQUIREMENTS

Construct a benchmark for intent–behavior misalignment analysis in agent skills. A real-world benchmark spanning multiple skill platforms and open source repositories is needed for unified and reproducible evaluation. It should preserve descriptive and executable content, including metadata, `SKILL.md`, `README`, auxiliary documentation, inline code, source files, scripts, and configuration files. The benchmark should annotate declared intents, execution behaviors, expected results, evidence locations, and three misalignment types: Unimplemented Intent, Undeclared Behavior, and Semantically Divergent Behavior. These annotations support systematic comparison across rule-based, static-analysis, semantic-analysis, and LLM-based methods.

Develop a multi-source method for intent and behavior extraction. The method should jointly analyze descriptive and executable content across heterogeneous sources. It extracts functional claims from descriptions and specification files, consolidates fragmented expressions into coherent intent units, and derives runtime behaviors and execution effects from inline code, source files, scripts, configuration files, and external API calls.

Develop a misalignment detection framework for functional fulfillment and composite behaviors. The framework should align declared intents with execution behaviors using behavior semantics, execution results, and contextual constraints, and distinguish the three misalignment types. For behaviors spanning functions, files, or execution steps, it should recover composite execution semantics from call relations, execution order, and data flow, and report the misalignment type, supporting evidence, relevant call chains, and risk explanations.

REFERENCES

[1] R. Xu and Y. Yan, "Agent skills for large language models: Architecture, acquisition, security, and the path forward," *arXiv preprint arXiv:2602.12430*, 2026.

[2] S. Wang, J. He, Y. Zhao, Y. Wang, K. Yu, and H. Wang, "" elementary, my dear watson." detecting malicious skills via neuro-symbolic reasoning across heterogeneous artifacts," *arXiv preprint arXiv:2603.27204*, 2026.

[3] Z. Guo, Z. Chen, X. Nie, J. Lin, Y. Zhou, and W. Zhang, "Skillprobe: Security auditing for emerging agent skill marketplaces via multi-agent collaboration," *arXiv preprint arXiv:2603.21019*, 2026.

[4] A. Dau, J. L. Guo, and N. Bui, "Docchecker: Bootstrapping code large language model for detecting and resolving code-comment inconsistencies," in *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*, 2024, pp. 187–194.